

IPC Experiment - Message Queues

Operating Systems Lab

Concept

- A Message Queue is a linked list of messages stored in kernel memory
- Identified by a unique key — both sender and receiver use the same key (12345)
- Messages have a type field (msg_type) allowing priority-based retrieval
- Queue persists until explicitly deleted with msgctl(IPC_RMID)
- Works between unrelated processes — no fork() or parent-child needed

Key System Calls

Function	Returns	Purpose
msgget(key, flags)	Queue ID or -1	Create or access a message queue
msgsnd(id, msg, size, flag)	0 or -1	Send a message to the queue
msgrcv(id, msg, size, type, flag)	0 or -1	Receive a message from the queue
msgctl(id, IPC_RMID, NULL)	0 or -1	Delete the message queue

Message Structure

Every message MUST start with a long int as the first field. This is mandatory.

```
struct my_msg {  
    long int msg_type;      // MUST be first, used for filtering  
    char some_text[MAX_TEXT]; // Your actual data  
};
```

Sender Program (msg_send.c)

What it does

- Creates the message queue using msgget() with key 12345
- Reads user input in a loop using fgets()
- Sets msg_type = 1 and copies input into the struct
- Sends via msgsnd() — size = strlen + 1 (includes null terminator)
- Breaks loop when user types "end"

Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/msg.h>

#define MAX_TEXT 512

struct my_msg {
    long int msg_type;
    char some_text[MAX_TEXT];
};

int main() {
    int msgid;
    struct my_msg message;
    char buffer[100];

    // Create message queue
    msgid = msgget((key_t)12345, 0666 | IPC_CREAT);
    if (msgid == -1) {
        perror("msgget failed");
        exit(1);
    }

    while (1) {
        printf("Enter some text: ");
        fgets(buffer, sizeof(buffer), stdin);

        message.msg_type = 1;
        strcpy(message.some_text, buffer);

        // Send only actual message length
        if (msgsnd(msgid, (void *)&message,
                    strlen(message.some_text) + 1, 0) == -1) {
            perror("msgsnd failed");
            exit(1);
        }

        if (strncmp(buffer, "end", 3) == 0) {
            break;
        }
    }

    return 0;
}
```

Compile & Run

```
gcc msg_send.c -o msg_send
./msg_send
```

Receiver Program (msg_receive.c)

What it does

- Accesses the same queue using the same key (12345)
- msgrcv() with msgtype=0 retrieves messages in FIFO order
- Prints each received message until "end" is received
- Deletes the queue with msgctl(IPC_RMID) on exit — cleanup is important!

Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/msg.h>

#define MAX_TEXT 512

struct my_msg {
    long int msg_type;
    char some_text[MAX_TEXT];
};

int main() {
    int msgid;
    struct my_msg message;

    // Access message queue
    msgid = msgget((key_t)12345, 0666 | IPC_CREAT);
    if (msgid == -1) {
        perror("msgget failed");
        exit(1);
    }

    while (1) {
        // Receive message (msgtype=0 means FIFO order)
        if (msgrcv(msgid, (void *)&message, MAX_TEXT, 0, 0) == -1) {
            perror("msgrcv failed");
            exit(1);
        }

        printf("Data received: %s", message.some_text);

        if (strncmp(message.some_text, "end", 3) == 0) {
            break;
        }
    }

    // Delete queue after use
    if (msgctl(msgid, IPC_RMID, NULL) == -1) {
        perror("msgctl failed");
    }

    return 0;
}
```

Compile & Run

```
gcc msg_receive.c -o msg_receive

# Run sender first (it creates the queue)
./msg_send

# In another terminal, run receiver
./msg_receive
```

Key Viva Points

- **msg_type = 0 in msgrcv:** retrieves messages in the order they were sent (FIFO)
- **msg_type > 0 in msgrcv:** retrieves first message of that specific type
- **IPC_CREAT flag:** creates the queue if it does not exist; ignored if it already exists
- **IPC_RMID:** only the receiver deletes the queue; sender does not
- Message size in msgsnd must NOT include the long int msg_type field
- Unlike pipes, message queue data is NOT lost if receiver is not ready yet
- Use ipcs -q in terminal to list existing message queues
- Use ipcrm -q <id> to manually delete a queue if program crashes

Pipes vs Message Queues (Quick Comparison)

Feature	Unnamed Pipe	Named Pipe	Message Queue
Processes	Parent-Child	Any	Any
Data type	Byte stream	Byte stream	Typed messages
Persistence	Temporary	Filesystem file	Kernel (until deleted)
Priority	No	No	Yes (msg_type)
Key syscall	pipe(), fork()	mkfifo()	msgget(), msgsnd()